



**SI-VISION
ACADEMY**

ALU SystemVerilog Verification Project

Prepared by:

Eng.Amir Mohamed

Table of Contents

1. Introduction.....	5
2. Design Specification	6
2.1 Logic References.....	8
2.2 TEST CASES.....	8
3. Design Implementation Overview	9
3.1 Design Requirement.....	9
3.2 Dut Code	10
4. Verification Enviroment Architecture	13
4.1 testbench architecture / Block diagram	13
4.2 flow of Testbench.....	13
4.3 Verification Plan	14
4.3.1 Interface	18
4.3.2 Interface Code	18
4.4 Transaction Class	23
4.4.1 Transaction Class Code.....	23
4.5 Generator Class	25
4.5.1 Generator Class Code	25
4.6 Driver Class	28
4.6.1 Driver Class Code	28
4.7 Monitor Class.....	31
4.7.1 Monitor Class Code	31
4.8 Scoreboard Class	33
4.8.1 Scoreboard Class Code	33
4.9 Environment Class.....	36
4.9.1 Environment Class Code.....	36
4.10 ALU Package	38
4.11 Top Module	39
4.11.1 Top Module Code	39
5. Coverage Results and Analysis.....	41
5.1 Coverage Results	41
5.2 Assertion Coverage Analysis	42
5.3 Code Coverage Analysis	43
5.4 Coverage Gap and Bug Analysis.....	44
5.4.1 Line Coverage Gap.....	44

5.4.2 Branch Coverage Gap	44
5.4.3 Condition Coverage Gap	45
5.4.3 Bug Analysis Summary	46
6. Simulation Result	47
7.Conclusion	49

List of Figures

Figure 1: ALU BLOCK	6
Figure 2: Total Coverage summary	41
Figure 3: Transactions.....	46
Figure 4:simulation results.....	47
Figure 5: simulation results.....	47
Figure 6: simulation results.....	48
Figure 7: simulation results.....	48

1. Introduction

In this project, a SystemVerilog OOP-based verification environment is used to functionally verify a signed 5-bit Arithmetic Logic Unit (ALU). Verifying that the ALU generates the right output for every valid operation mode specified in the design specification is the primary goal of the project.

The ALU generates a signed 6-bit output C and allows arithmetic and logical operations utilizing two signed 5-bit operands, A and B. The control signals ALU_en, a_en, b_en, a_op, and b_op determine the chosen operation.

SystemVerilog classes, interface, assertions, functional coverage, directed tests, constrained-random tests, monitor, and scoreboard were used to create the verification environment. The scoreboard contrasts the expected output produced by a golden reference model with the actual DUT output.

2. Design Specification

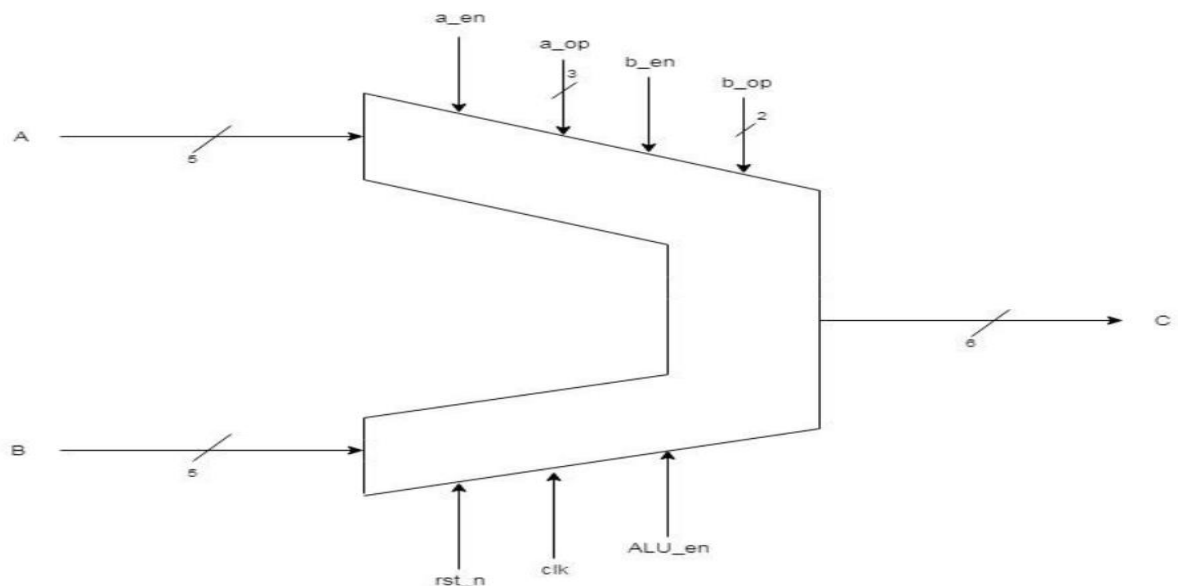


FIGURE 1: ALU BLOCK

This Section Discusses the different inputs and outputs of the ALU block (Figure 1) with a brief description of each. The following table will contain the overall system I/O:

Signal Name	Type	Width (Bits)	Description
A	Input	5	Signed 5 bits input data
B	Input	5	Signed 5 bits input data
a_en	Input	1	Enable signal for a_op operations
a_op	Input	3	3 bits defining a set of operations to be performed
b_en	Input	1	Enable signal for b_op operations
b_op	Input	2	2 bits defining a set of operations to be performed
rst_n	Input	1	Asynchronous Active Low reset signal
clk	Input	1	System input Clock Signal
Alu_en	Input	1	The overall system Enable signal
C	Output	6	Signed 6 bits output data

1. **Rst_n:** a reset signal that clears any pending errors or events and brings a system to normal condition or an initial state.
2. **ALU_en:** Enable signal to the system.
3. **A_en:** When this enable signal is active high, only the set of a_op (Spec #7) operations are performed.
4. **B_en:** when this enable signal is active high, only a set of b_op (Spec #8) operations are performed.
5. **Input A:** This is a 5 bits signed input bus to the ALU ranging from (-15 to +15) on which operations will be performed.
6. **Input B:** This is a 5 bits signed input the ALU ranging from (-15 to +15) on which operations will be performed.
7. **Output C:** This is a 6 bits signed Output bus to the ALU ranging from (-31 to + 31) which contains the results of ALU operations.

8. **a_op**: it is 3 unsigned bus representing an encoding for a set of operations. Each value refers to a specific function for the ALU to perform. The Following table (Table 2) illustrates these functions with a brief description of each, putting into consideration the following 2 constraints:

- These functions are performed if and only if the a_en is active-high and b_en is active low.
- The values with a “Null” operation refer to an illegal input for the system.

Sub_Spec	A-op	C(Function)	Description
8.a	0	A + B	The Summation of the inputs
8.b	1	A - B	Subtracting Input B from A
8.c	2	A “XOR” B	Bitwise “XOR” logic operation
8.d	3	A & B	Bitwise “AND” logic operation
8.e	4	A & B	Bitwise “AND” logic operation
8.f	5	A B	Bitwise “OR” logic operation
	6	A “XNOR” B	Bitwise “XNOR” logic operation
	7	Null	The System is not permitted to take this value

9. **b_op (SET 1)**: It is 3-bit unsigned bus representing an encoding for a set of operations. Each value refers to a specific function for the ALU to perform. There are 2 different sets of these operations depending on the enable signals (a_en & b_en). The following table (Table 3) illustrates the first set of functions with a brief description of each, putting into consideration the following 2 constraints:

- These functions are performed if and only if the b_en is active-high and a_en is active low.
- The values with a “Null” operation refer to an illegal input for the system.

Sub_Specs	B_op (set 1)	C (Function)	Description
9.a	0	A “AND” B	The summation compliment of the inputs
9.b	1	A + B	The summation of the inputs
9.c	2	A + B	The summation of the inputs
	3	Null	The system is not permitted to take this value

10. **b_op (SET 2)**: These are the second set of b_operations. The following table (Table 3) illustrates these functions with a brief description of each, putting into consideration the following 2 constraints:

- These functions are performed **if and only if both** the b_en and a_en are active high.

Sub_Specs	b_op (Set 2)	C (Function)	Description
10.a	0	A “XOR” B	Bitwise “XOR” logic operation
10.b	1	A “XNOR” B	
10.c	2	A - 1	
10.d	3	B + 2	

2.1 Logic References

As observed in section 2, various operations are based on standard logic gates such as XOR, XNOR, etc... The logical truth table of these gates are stated in table 5, as a reference during the output evaluation stage.

INPUTS		GATES				
A(1bit)	B(1bit)	AND	NAND	OR	XOR	XNOR
0	0	0	1	0	0	1
0	1	0	1	1	1	0
1	0	0	1	1	1	0
1	1	1	0	1	0	1

2.2 TEST CASES

In this section we are going to handle the functional verification using selected test cases to ensure that the system is functioning as per required in all cases within the specified specs extracted in the section above. In each test case the following aspects are considered:

1. Derivation of Inputs to create the needed environment for testing.
2. Monitoring the output as a result of the tested environment.
3. Checking the output is as expected.

3. Design Implementation Overview

3.1 Design Requirement

SPEC ID	SPEC Description
ALU_01	clk: System clock used to register the ALU output C on the positive edge.
ALU_02	rst_n: Asynchronous active-low reset. When rst_n = 0, output C must be cleared to zero.
ALU_03	A and B: Signed 5-bit input operands used for arithmetic and logical ALU operations.
ALU_04	C: Signed 6-bit registered output used to store the ALU operation result.
ALU_05	ALU_en: Overall ALU enable. When ALU_en = 0, the expected output C is zero.
ALU_06	a_en: Enable signal for a_op operation set. a_op operations are selected when a_en = 1 and b_en = 0.
ALU_07	a_op: 3-bit opcode for A-operation set. Legal values in A-only mode are 0 to 6; value 7 is illegal/null.
ALU_08	When a_en = 1 and b_en = 0, a_op = 0 shall perform A + B.
ALU_09	When a_en = 1 and b_en = 0, a_op = 1 shall perform A - B.
ALU_10	When a_en = 1 and b_en = 0, a_op = 2 shall perform A XOR B.
ALU_11	When a_en = 1 and b_en = 0, a_op = 3 shall perform A AND B.
ALU_12	When a_en = 1 and b_en = 0, a_op = 4 shall perform A AND B.
ALU_13	When a_en = 1 and b_en = 0, a_op = 5 shall perform A OR B.
ALU_14	When a_en = 1 and b_en = 0, a_op = 6 shall perform A XNOR B.
ALU_15	b_en: Enable signal for b_op operation sets. b_op set 1 is selected when a_en = 0 and b_en = 1.
ALU_16	b_op: 2-bit opcode. In B-only mode, legal values are 0 to 2; value 3 is illegal/null.
ALU_17	When a_en = 0 and b_en = 1, b_op = 0 shall perform NAND operation $\sim(A \& B)$.
ALU_18	When a_en = 0 and b_en = 1, b_op = 1 shall perform A + B.
ALU_19	When a_en = 0 and b_en = 1, b_op = 2 shall perform A + B.
ALU_20	When a_en = 1 and b_en = 1, b_op set 2 shall be selected and all b_op values 0 to 3 are legal.
ALU_21	When a_en = 1 and b_en = 1, b_op = 0 shall perform A XOR B.
ALU_22	When a_en = 1 and b_en = 1, b_op = 1 shall perform A XNOR B.
ALU_23	When a_en = 1 and b_en = 1, b_op = 2 shall perform A - 1.
ALU_24	When a_en = 1 and b_en = 1, b_op = 3 shall perform B + 2.
ALU_25	When a_en = 0 and b_en = 0, no operation is selected and output C shall be zero.
ALU_26	Logical operation results shall be sign-extended from 5 bits to the 6-bit signed output C.
ALU_27	The ALU shall be verified using directed tests to cover all legal operation bins and boundary operand values.
ALU_28	The ALU shall be verified using constrained-random tests to improve stimulus diversity.
ALU_29	Assertions shall check reset behavior, ALU disabled behavior, illegal opcode prevention, and output range.
ALU_30	Functional coverage shall cover reset, ALU enable, enable mode combinations, opcode bins, and A/B boundary value bins.

3.2 Dut Code

```
//firstly, time_unit = 1ns, time_precision = 1ps
`timescale 1ns/1ps

module alu (
    //5-bits signed input A,B  [-16 to +15]
    input logic signed [4:0] A,
    input logic signed [4:0] B,
    //Enable signals for a_op operations
    input logic          a_en,
    //3 bits defining a set of operations to be performed
    input logic          [2:0] a_op,
    //Enable signals for b_op operations
    input logic          b_en,
    //2 bits defining a set of operations to be performed
    input logic          [1:0] b_op,
    //Asynchronous Active Low reset signal ( reset=0 -> reset active, reset=1
-> normal operation )
    input logic          rst_n,
    //System input Clock Signal
    input logic          clk,
    //The overall system Enable signal
    input logic          ALU_en,
    //6-bits Signed output data [15+15=30, -16+ (-16)=-32]
    output logic signed [5:0] C
);

/*automatic function for extension 5-bit to 6-bits
it takes 5-bits and returned signed 6-bits
We need this function because operations like
in_a ^ in_b,in_a & in_b,in_a | in_b ==> their output is 5bits
while the C output is signed 6-bits
so we need signed extension
*/
function automatic logic signed [5:0] ext5(input logic [4:0] value);
    //concatenations takes signed bit [4] and put it firstly
    //if value is +ve like value=5'b00101 (+5)
    //value[4]=0, {value[4],value} = 6'b000101

    //if value is -ve like value=5'b11100 (-4)
    //value[4]=1, {value[4],value}=6'b111100

    ext5 = $signed({value[4], value});
endfunction

//automatic function for calculation the result
```

```

function automatic logic signed [5:0] calc_result(
    //inputs of the functions
    input logic signed [4:0] in_a,
    input logic signed [4:0] in_b,
    input logic             in_a_en,
    input logic             [2:0] in_a_op,
    input logic             in_b_en,
    input logic             [1:0] in_b_op,
    input logic             in_alu_en
);

    //if ALU_en=0 then output=0
    if (!in_alu_en) begin
        calc_result = '0; //all bits are filled with zero
    end
    //Case1
    //if (a_en=1, b_en=0)
    //in this case we use a_op
    else if (in_a_en && !in_b_en) begin
        case (in_a_op)
            3'd0: calc_result = in_a + in_b;
            3'd1: calc_result = in_a - in_b;
            3'd2: calc_result = ext5(in_a ^ in_b); //XOR
            3'd3: calc_result = ext5(in_a & in_b); //AND
            3'd4: calc_result = ext5(in_a & in_b); //AND
            3'd5: calc_result = ext5(in_a | in_b); //OR
            3'd6: calc_result = ext5(~(in_a ^ in_b)); //XNOR
            default: calc_result = '0; //Null,illegal (a_op)
        endcase
    end

    //Case2
    //if (a_en=0, b_en=1)
    //in this case we use b_op set 1
    else if (!in_a_en && in_b_en) begin
        case (in_b_op)
            2'd0: calc_result = ext5(~(in_a & in_b)); //Complement of the
input which is AND
            2'd1: calc_result = in_a + in_b; //addition
            2'd2: calc_result = in_a + in_b; //addition
            default: calc_result = '0;
        endcase
    end
    //Case3
    //if (a_en=1, b_en=1)
    //in this case we use b_op set 2
    else if (in_a_en && in_b_en) begin

```

```

        case (in_b_op)
            2'd0: calc_result = ext5(in_a ^ in_b); //XOR
            2'd1: calc_result = ext5(~(in_a ^ in_b)); //XNOR
            2'd2: calc_result = in_a - 6'sd1; //A-(signed decimal 1)
            2'd3: calc_result = in_b + 6'sd2; //B+2
        endcase
    end
    //Case4
    //if (a_en=0, b_en=0)
    //expected output is 0
    else begin
        calc_result = '0;
    end
endfunction

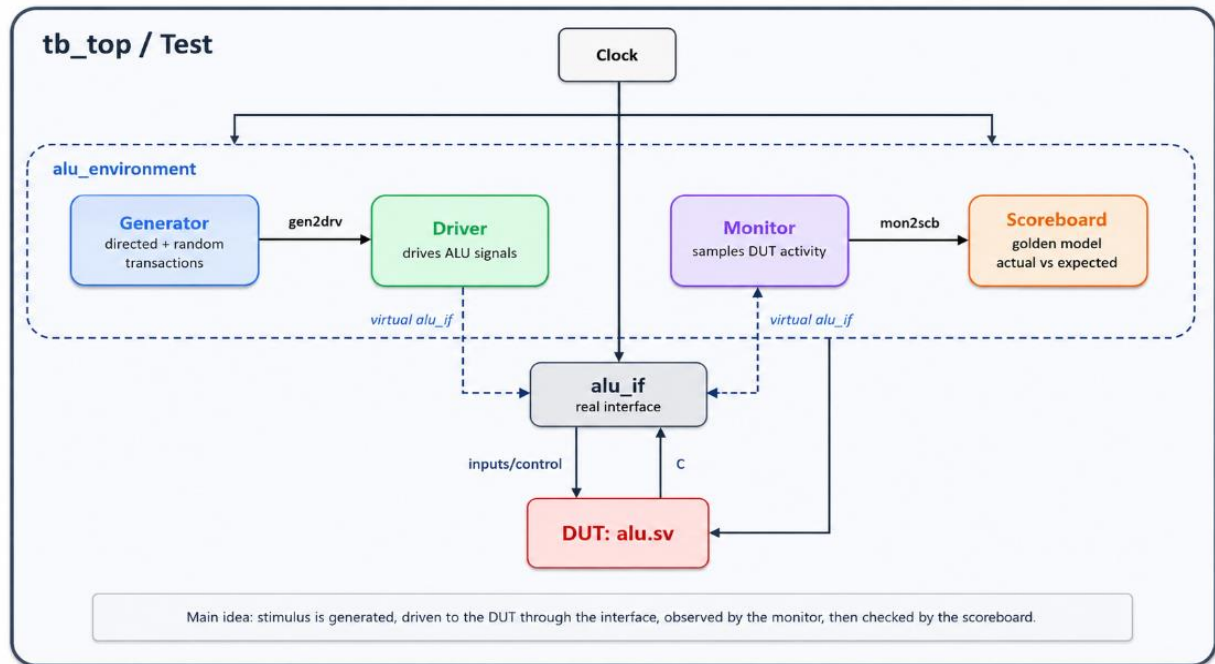
//Sequential Block
always_ff @(posedge clk or negedge rst_n) begin
    //Active-Low
    if (!rst_n) begin
        C <= '0;
    end
    else begin
        C <= calc_result(A, B, a_en, a_op, b_en, b_op, ALU_en);
    end
end

endmodule

```

4. Verification Environment Architecture

4.1 testbench architecture / Block diagram



4.2 flow of Testbench

The verification environment is based on a SystemVerilog OOP structure. It contains a transaction class, generator, driver, monitor, scoreboard, environment, interface, and top module.

The main verification flow is:

Generator → Driver → Interface → DUT → Monitor → Scoreboard

1. The generator makes ALU transactions.
2. These transactions are sent to the driver through a mailbox.
3. The driver converts each transaction into signal-level activity and drives the ALU interface.
4. The DUT receives the input signals through the interface and produces the output C.
5. The monitor observes the interface signals after the DUT output is updated, it captures the applied inputs, control signals, and actual output, then sends the observed transaction to the scoreboard.
6. The scoreboard calculates the expected output using a golden reference model and compares it with the actual DUT output.

4.3 Verification Plan

The verification plan was formed to make sure that all ALU requirements are checked using appropriate verification methods. The plan includes directed testing, constrained-random testing, assertions, functional coverage, and scoreboard checking.

Throughout this verification plan includes both legal operation modes and important boundary cases. Illegal opcode values are not generated as normal legal stimulus because they are classified as invalid operation cases.

SPEC ID	SPEC Description	Requirement (chk/cov/test)	component	Comment	status
ALU_01	clk: System clock used to register the ALU output C on the positive edge.	Test / Check	tb_top, alu_if, alu_driver, alu_monitor	Clock generated in tb_top. Driver drives on negedge, DUT updates on posedge, monitor samples after posedge.	Done
ALU_02	rst_n: Asynchronous active-low reset. When rst_n = 0, output C must be cleared to zero.	Check / Assertion / Coverage	alu_driver, alu_if, alu	Driver applies reset before tests. reset_p assertion checks C == 0 during reset. cp_rst_n covers asserted/released.	Done
ALU_03	A and B: Signed 5-bit input operands used for arithmetic and logical ALU operations.	Test / Coverage	alu_transaction, alu_generator, alu_if	Constrained-random stimulus covers signed operand range; directed tests include boundary values such as -16, 0, and 15.	Done
ALU_04	C: Signed 6-bit registered output used to store the ALU operation result.	Check / Assertion	alu, alu_monitor, alu_scoreboard, alu_if	Monitor captures actual C. Scoreboard compares actual against golden model. range_p assertion checks signed 6-bit output range.	Done
ALU_05	ALU_en: Overall ALU enable. When ALU_en = 0, expected output C is zero.	Check / Coverage / Test	alu_transaction, alu_generator, alu_scoreboard, alu_if	Directed and random tests include ALU_en = 0. alu_disable_p assertion and scoreboard golden model check C = 0.	Done
ALU_06	a_en: Enable signal for a_op operation set. a_op operations selected when	Test / Coverage	alu_generator, alu_if	Enable-mode coverpoint cp_enable_mode covers a_only, b_only, both, and none modes.	Done

	a_en = 1 and b_en = 0.				
ALU_07	a_op: 3-bit opcode. Legal values in A-only mode are 0 to 6; value 7 is illegal/null.	Assertion / Coverage / Test	alu_transaction, alu_if, alu_generator	Constraint prevents illegal a_op = 7 in A-only mode. illegal_aop_p assertion checks that illegal opcode is not applied.	Done
ALU_08	A-only mode a_op = 0 performs A + B.	Check / Coverage / Directed Test	alu_generator, alu_scoreboard, alu	Directed test covers a_op 0. Golden model checks A+B result.	Done
ALU_09	A-only mode a_op = 1 performs A - B.	Check / Coverage / Directed Test	alu_generator, alu_scoreboard, alu	Directed test covers a_op 1. Golden model checks A-B result.	Done
ALU_10	A-only mode a_op = 2 performs A XOR B.	Check / Coverage / Directed Test	alu_generator, alu_scoreboard, alu	Directed test covers XOR. Scoreboard sign-extends logical result to 6-bit signed.	Done
ALU_11	A-only mode a_op = 3 performs A AND B.	Check / Coverage / Directed Test	alu_generator, alu_scoreboard, alu	Directed test covers AND operation.	Done
ALU_12	A-only mode a_op = 4 performs A AND B.	Check / Coverage / Directed Test	alu_generator, alu_scoreboard, alu	Directed test covers duplicate AND operation as specified.	Done
ALU_13	A-only mode a_op = 5 performs A OR B.	Check / Coverage / Directed Test	alu_generator, alu_scoreboard, alu	Directed test covers OR operation.	Done
ALU_14	A-only mode a_op = 6 performs A XNOR B.	Check / Coverage / Directed Test	alu_generator, alu_scoreboard, alu	Directed test covers XNOR using $\sim(A \wedge B)$.	Done
ALU_15	b_en selects b_op operation set. B-only mode is selected when a_en = 0 and b_en = 1.	Test / Coverage	alu_generator, alu_if	cp_enable_mode covers B-only mode. Directed tests include b_op set 1.	Done
ALU_16	In B-only mode, legal b_op values are 0 to 2; value 3 is illegal/null.	Assertion / Coverage / Test	alu_transaction, alu_if, alu_generator	Constraint prevents b_op = 3 in B-only mode. illegal_bop_p assertion checks illegal opcode prevention.	Done
ALU_17	B-only mode b_op = 0 performs NAND $\sim(A \& B)$.	Check / Coverage / Directed Test	alu_generator, alu_scoreboard, alu	Directed test covers b_op 0 set 1. Golden model checks NAND operation.	Done

ALU_18	B-only mode b_op = 1 performs A + B.	Check / Coverage / Directed Test	alu_generator, alu_scoreboard, alu	Directed test covers b_op 1 set 1. Golden model checks A+B result.	Done
ALU_19	B-only mode b_op = 2 performs A + B.	Check / Coverage / Directed Test	alu_generator, alu_scoreboard, alu	Directed test covers b_op 2 set 1. Golden model checks A+B result.	Done
ALU_20	Both-enable mode is selected when a_en = 1 and b_en = 1; all b_op values 0 to 3 are legal.	Test / Coverage	alu_generator, alu_if	cp_b_op_both covers all b_op values in both-enable mode.	Done
ALU_21	Both-enable mode b_op = 0 performs A XOR B.	Check / Coverage / Directed Test	alu_generator, alu_scoreboard, alu	Directed test covers both-mode b_op 0. Golden model checks XOR.	Done
ALU_22	Both-enable mode b_op = 1 performs A XNOR B.	Check / Coverage / Directed Test	alu_generator, alu_scoreboard, alu	Directed test covers both-mode b_op 1. Golden model checks XNOR.	Done
ALU_23	Both-enable mode b_op = 2 performs A - 1.	Check / Coverage / Directed Test	alu_generator, alu_scoreboard, alu	Directed test covers both-mode b_op 2. Golden model checks A-1.	Done
ALU_24	Both-enable mode b_op = 3 performs B + 2.	Check / Coverage / Directed Test	alu_generator, alu_scoreboard, alu	Directed test covers both-mode b_op 3. Golden model checks B+2.	Done
ALU_25	When a_en = 0 and b_en = 0, no operation is selected and output C shall be zero.	Check / Coverage / Directed Test	alu_generator, alu_scoreboard, alu_if	Directed no-operation test included. cp_enable_mode covers none mode.	Done
ALU_26	Logical operation results are sign- extended from 5 bits to 6-bit signed output.	Check	alu, alu_scoreboard	ext5 function used in DUT and golden model for logical operations. Scoreboard compares against sign-extended expected output.	Done
ALU_27	Directed tests shall cover all legal operation bins and boundary operand values.	Test / Coverage	alu_generator, alu_if	Directed tests cover all legal opcode bins and boundary operands. Covergroup report shows all bins covered.	Done

ALU_28	Constrained-random tests shall improve stimulus diversity.	Test	alu_transaction, alu_generator	Generator sends 100 constrained-random transactions after directed tests. Legal opcode constraints are applied.	Done
ALU_29	Assertions shall check reset behavior, disabled ALU behavior, illegal opcode prevention, and output range.	Assertion	alu_if	Assertions reset_p, alu_disable_p, illegal_aop_p, illegal_bop_p, and range_p all passed with 100% assertion coverage.	Done
ALU_30	Functional coverage shall cover reset, ALU enable, enable modes, opcode bins, and A/B boundary values.	Coverage	alu_if	Covergroup alu_functional_cg reached 100% with all 32 bins covered.	Done
ALU_31	Scoreboard shall compare DUT output with reference golden model.	Check	alu_monitor, alu_scoreboard	Monitor sends observed transaction to scoreboard. Scoreboard reported PASS = 118 and FAIL = 0.	Done
ALU_32	Coverage closure shall explain remaining code coverage gaps.	Coverage / Report	coverage_report, Synopsys URG report	Remaining uncovered line/branch/condition items are illegal/default or failure-handling paths; justify or exclude during coverage closure.	Done

4.3.1 Interface

The interface carries all ALU signals, including inputs, output, clock, and reset. It is used to connect the DUT with the verification components. The interface also contains assertions and functional coverage.

4.3.2 Interface Code

```
`timescale 1ns/1ps

interface alu_if(input logic clk);

    //signals
    logic signed [4:0] A;
    logic signed [4:0] B;
    logic              a_en;
    logic              [2:0] a_op;
    logic              b_en;
    logic              [1:0] b_op;
    logic              rst_n;
    logic              ALU_en;
    logic signed [5:0] C;

    //dut modport
    modport dut (
        input  A,
        input  B,
        input  a_en,
        input  a_op,
        input  b_en,
        input  b_op,
        input  rst_n,
        input  clk,
        input  ALU_en,
        output C
    );

    //=====Assertions=====

    property reset_p;
        @(posedge clk)
        //if reset(Asynchronous) is active then C must equal to zero @(+vedge)
        (!rst_n) |-> (C == '0); //using overlapped implication (happens at
same time)
    endproperty
```

```

assert property (reset_p)
else $error("ASSERT FAIL: Reset active but C != 0");

//property for ALU disable
property alu_disable_p;
    @(posedge clk)
    //if reset is active, stop checking this property
    disable iff (!rst_n)
    //non-overlapped implication
    (!ALU_en) | => (C == '0); //if ALU_en=0, next cycle C=0
endproperty

assert property (alu_disable_p)
else $error("ASSERT FAIL: ALU_en is low but C did not become 0");

//illegal property for a_op
property illegal_aop_p;
    @(posedge clk)
    disable iff (!rst_n)
    //if ALU_en is 1 (enabled) and mode is a only then a_op can't be 7
    //because specs says that a_op = 7 is Null/illegal in a_op mode
    (ALU_en && a_en && !b_en) | -> (a_op != 3'd7);
endproperty

assert property (illegal_aop_p)
else $error("ASSERT FAIL: Illegal a_op = 7 detected in A-only mode");

//illegal property for b_op
property illegal_bop_p;
    @(posedge clk)
    disable iff (!rst_n)
    //if ALU_en is 1 (enabled) and mode is b only then b_op can't be 3
    //because specs says that b_op = 3 is Null/illegal in b_op set 1 mode
    (ALU_en && !a_en && b_en) | -> (b_op != 2'd3);
endproperty

assert property (illegal_bop_p)
else $error("ASSERT FAIL: Illegal b_op = 3 detected in B-only mode");

//Output Range Property
property range_p;
    @(posedge clk)
    disable iff (!rst_n)
    //output should be between -32 to +31 (signed 6-bits)
    (C inside {[-32:31]});

```

```

endproperty
//since this is signed 6-bits output
//normally the output is between the specified range
//this is calll sanity check more than functionality check
assert property (range_p)
else $error("ASSERT FAIL: Output out of signed 6-bit range");

//=====Functional
Coverage=====
//functional coverage, to ask whether the tests has covered the specified
cases or not
//@each clock cycle, see the signals and decide that the bins is covered
or not
covergroup alu_functional_cg @(posedge clk);

    option.per_instance = 1; //calculate coverage for each instance from
covergroup
    //in our project we have one instance ( alu_cg )

    //coverpoint on reset
    cp_rst_n: coverpoint rst_n {
        bins asserted = {0}; //rst_n = 0 -> reset asserted
        bins released = {1}; //rst_n = 1 -> reset released
    }

    //coverpoint on ALU_en
    //do not take sample unless reset is not active
    cp_ALU_en: coverpoint ALU_en iff (rst_n) {
        bins disabled = {0}; //ALU_en = 0 -> disabled
        bins enabled = {1}; //ALU_en = 1 -> enabled
    }

    //coverpoint on enable modes
    //concatenation between {a_en, b_en}
    cp_enable_mode: coverpoint {a_en, b_en} iff (rst_n && ALU_en) {
        bins none = {2'b00}; //a_en = 0, b_en = 0 -> 2'b00 (none)
        bins a_only = {2'b10}; //a_en = 1, b_en = 0 -> 2'b00 (use a_op
table)
        bins b_only = {2'b01}; //a_en = 0, b_en = 1 -> 2'b00 (use b_op set1
table)
        bins both = {2'b11}; //a_en = 1, b_en = 1 -> 2'b00 (use b_op set2
table)
    }

    //coverpoint on a_op
    //sample a_op only when -> rst_n=1, ALU_en=1, a_en=1, b_en=0
    cp_a_op_a_only: coverpoint a_op iff (rst_n && ALU_en && a_en && !b_en)
{

```

```

        bins op0 = {3'd0};
        bins op1 = {3'd1};
        bins op2 = {3'd2};
        bins op3 = {3'd3};
        bins op4 = {3'd4};
        bins op5 = {3'd5};
        bins op6 = {3'd6};
        illegal_bins illegal_op7 = {3'd7}; //illegal bin
    }

    //coverpoint on b_op
    //sample a_op only when -> rst_n=1, ALU_en=1, a_en=0, b_en=1
    cp_b_op_b_only: coverpoint b_op iff (rst_n && ALU_en && !a_en && b_en)
{
        bins op0 = {2'd0};
        bins op1 = {2'd1};
        bins op2 = {2'd2};
        illegal_bins illegal_op3 = {2'd3}; //illegal bin
    }

    //coverpoint on b_op in both modes when (a_en=1, b_en=1)
    cp_b_op_both: coverpoint b_op iff (rst_n && ALU_en && a_en && b_en) {
        bins op0 = {2'd0};
        bins op1 = {2'd1};
        bins op2 = {2'd2};
        bins op3 = {2'd3};
    }

    //coverpoint for A if reset is released and ALU enabled
    cp_A_values: coverpoint A iff (rst_n && ALU_en) {
        bins min      = {-16}; //cover least
        bins zero     = {0};    //cover zero
        bins max      = {15};   //cover max positive
        bins negative = {[-15:-1]}; //cover negative value
        bins positive = {[1:14]}; //cover positive value
    }

    //coverpoint for B if reset is released and ALU enabled
    cp_B_values: coverpoint B iff (rst_n && ALU_en) {
        bins min      = {-16};
        bins zero     = {0};
        bins max      = {15};
        bins negative = {[-15:-1]};
        bins positive = {[1:14]};
    }

endgroup

```

```
//make instance from covergroup called alu_cg  
alu_functional_cg alu_cg = new();  
  
endinterface
```

4.4 Transaction Class

The transaction class represents one ALU test item. It contains the randomized operands, control signals, actual output, and expected output. The transaction is used to transfer data between the generator, driver, monitor, and scoreboard.

4.4.1 Transaction Class Code

```
class alu_transaction;
    //transaction is the data transfered between blocks
    //rand variable to use randomization
    rand logic signed [4:0] A;
    rand logic signed [4:0] B;

    rand logic          ALU_en;
    rand logic          a_en;
    rand logic          [2:0] a_op;
    rand logic          b_en;
    rand logic          [1:0] b_op;

    //store actual output from DUT
    //monitor will fill it as (tr.actual = vif.C)
    logic signed [5:0] actual;
    //Store expected from specs from Scoreboard Calculation
    //tr.expected = golden_model (tr);
    logic signed [5:0] expected;
    //then make comparison between actual and expected

    //constraint for inputs A and B ranges
    constraint input_range_c {
        A inside {[-16:15]};
        B inside {[-16:15]};
    }

    //constraint on ALU_en
    constraint enable_c {
        ALU_en dist {1 := 9, 0 := 1}; //distribution
        //ALU_en =1 weight 9, 90% enabled
        //ALU_en =0 weight 1, 10% disabled
    }

    //constraint for preventing illegal opcodes in random tests
    constraint legal_opcode_c {

        if (ALU_en && a_en && !b_en)
            //if ALU enabled and a_en=1 and b_en=0 (A-only mode)
            a_op inside {[3'd0:3'd6]}; //a_op inside from 0 to 6 and prevent 7

        if (ALU_en && !a_en && b_en)
```

```

        //if ALU enabled and a_en=0 and b_en=1 (B-only mode)
        b_op inside {[2'd0:2'd2]}; //a_op inside from 0 to 2 and prevent 3
    }

    //Constructor new
    function new();

        A          = 0;
        B          = 0;
        ALU_en      = 0;
        a_en        = 0;
        a_op        = 0;
        b_en        = 0;
        b_op        = 0;
        actual      = 0;
        expected    = 0;

    endfunction

    //function to return string
    //we will use it for debugging and display
    function string to_string();

        return $sformatf("A=%0d B=%0d ALU_en=%0b a_en=%0b a_op=%0d b_en=%0b
b_op=%0d expected=%0d actual=%0d",
                        A, B, ALU_en, a_en, a_op, b_en, b_op, expected,
actual);

    endfunction

endclass

```


4.5 Generator Class

The generator makes both directed and constrained-random transactions. Directed tests are used to make sure that all significant operation modes and boundary values are tested. Random tests are used to increase verification confidence by testing different input combinations.

4.5.1 Generator Class Code

```
class alu_generator;
    // we will create ALU transaction
    // Send them to the driver

    //mailbox generator to driver
    //generator will put inside the transaction
    //& the driver will take the transaction from this mailbox
    mailbox #(alu_transaction) gen2drv;

    int random_count;// integer to calculate the number of random trans.

    //constructor new (called whenever we make object from generator)
    function new(mailbox #(alu_transaction) gen2drv, int random_count = 100);
    //constructor take 2 i/p which are mailbox gen2dr and random_count
        this.gen2drv      = gen2drv;
        //store mailbox that comes from env inside generator
        this.random_count = random_count;

    endfunction

    //task for specific test cases manually
    task send_directed(
        logic signed [4:0] A,
        logic signed [4:0] B,
        logic              ALU_en,
        logic              a_en,
        logic [2:0] a_op,
        logic              b_en,
        logic [1:0] b_op
    );

        alu_transaction tr; // handle tr

        tr = new(); // transaction object

        //put argument A in field A inside transaction
        tr.A      = A;
        //same as previous one
```

```

tr.B      = B;
tr.ALU_en = ALU_en;
tr.a_en   = a_en;
tr.a_op   = a_op;
tr.b_en   = b_en;
tr.b_op   = b_op;

//put this transaction into gen2drv mailbox
gen2drv.put(tr);
//after this the driver will take this transaction (gen2drv.get(tr))
//take same transaction and start to drive signals
endtask

//when enviroment makes gen.run() then generator starts
//run will make 2 things, 1.send directed test, 2.send random test
task run();

    alu_transaction tr;//transaction handle will use it in random_tests

    // A enable only
    //ALU_en=1, a_en=1, b_en=0
    send_directed(5'sd7,      5'sd3,      1'b1, 1'b1, 3'd0, 1'b0, 2'd0);
    send_directed(5'sd7,      5'sd3,      1'b1, 1'b1, 3'd1, 1'b0, 2'd0);
    send_directed(5'sd10,     5'sd5,      1'b1, 1'b1, 3'd2, 1'b0, 2'd0);
    send_directed(-5'sd4,     5'sd6,      1'b1, 1'b1, 3'd3, 1'b0, 2'd0);
    send_directed(-5'sd4,     5'sd6,      1'b1, 1'b1, 3'd4, 1'b0, 2'd0);
    send_directed(5'sd8,      5'sd2,      1'b1, 1'b1, 3'd5, 1'b0, 2'd0);
    send_directed(5'sd8,      5'sd2,      1'b1, 1'b1, 3'd6, 1'b0, 2'd0);

    // signed boundaries
    send_directed(5'sb10000, 5'sd0,      1'b1, 1'b1, 3'd0, 1'b0, 2'd0);//-
16+0=-16 expected
    send_directed(5'sd0,      5'sb10000, 1'b1, 1'b1, 3'd0, 1'b0, 2'd0);

    // B enable only
    //ALU_en=1, a_en=0, b_en=1
    send_directed(5'sd7,      5'sd3,      1'b1, 1'b0, 3'd0, 1'b1, 2'd0);
    send_directed(5'sd7,      5'sd3,      1'b1, 1'b0, 3'd0, 1'b1, 2'd1);
    send_directed(-5'sd5,     5'sd4,      1'b1, 1'b0, 3'd0, 1'b1, 2'd2);

    // both enables
    //ALU_en=1, a_en=1, b_en=1
    send_directed(5'sd10,     5'sd5,      1'b1, 1'b1, 3'd0, 1'b1, 2'd0);
    send_directed(5'sd10,     5'sd5,      1'b1, 1'b1, 3'd0, 1'b1, 2'd1);
    send_directed(-5'sd8,     5'sd0,      1'b1, 1'b1, 3'd0, 1'b1, 2'd2);
    send_directed(5'sd0,      -5'sd3,     1'b1, 1'b1, 3'd0, 1'b1, 2'd3);

```

```

// disabled and no operation
//ALU disabled
send_directed(5'sd7,      5'sd3,      1'b0, 1'b1, 3'd0, 1'b0, 2'd0);
// No operation is selected
send_directed(5'sd7,      5'sd3,      1'b1, 1'b0, 3'd0, 1'b0, 2'd0);

//Random testing
repeat (random_count) begin

    tr = new(); //new transaction

    //randomization for field of rand in alu_transactions but with
respect to constraints
    if (!tr.randomize()) begin
        $fatal(1, "Randomization failed"); //if randomization failed
simulation stops
        // $fatal is stronger than error means simulation has a big
problem so stop it
    end
    // if randomization succeeded then send this transaction to driver
    gen2drv.put(tr);

end
endtask
endclass

```

4.6 Driver Class

The driver accepts transactions from the generator through the gen2drv mailbox. It drives the transaction fields to the interface signals at the proper simulation time.

4.6.1 Driver Class Code

```
class alu_driver;
//takes transactions from generator and drive transactions values to the
interface

    //real interface on alu_if and it is made in tb_top (alu_if alu_vif(clk));

    //virtual interface as driver class is not module and thus we can't
instantiate from interface
    virtual alu_if vif; //vif inside driver points to alu_vif which is inside
top
    //and from this we can write vif.A <= tr.A;

    //mailbox between generator to driver
    mailbox #(alu_transaction) gen2drv;
    //generator makes gen2drv.put(tr); while driver makes gen2drv.get(tr);

    //constructor is called when env makes -> drv=new(vif, ge2drv);
    //gives driver 2 things 1.virtual interface handle, mailbox connected with
generator
    function new(virtual alu_if vif, mailbox #(alu_transaction) gen2drv);
        //this.sth means argument inside the constructor
        this.vif      = vif; //store virtual interface which comes from env
inside driver
        this.gen2drv = gen2drv;

    endfunction

    //task for making reset for DUT from the beggining of the simulation
    //driver is resposible for it bec driver is responsible for driving
signals to DUT

    task reset();

        vif.A      <= '0;
        vif.B      <= '0;
        vif.a_en   <= 1'b0;
        vif.a_op   <= '0;
        vif.b_en   <= 1'b0;
        vif.b_op   <= '0;
```

```

vif.ALU_en <= 1'b0;

vif.rst_n <= 1'b1; //initially inactive

@(posedge vif.clk); //wait till one rising edge of clock

vif.rst_n <= 1'b0; //reset active means design C<=0
repeat (3) @(posedge vif.clk); //make reset active for 3 cycles
// not specied no. as 3, but to make sure that design is in reset
mode and assertions and coverage see reset asserted

vif.rst_n <= 1'b1; // then release reset

@(posedge vif.clk); //wait one clock cylce after releasing reset
//bec to give the design opportunity to be stable before driving
transactions

endtask

task run(int count); //task run of driver, takes input count
//count is the number of transactions that driver receives
/*in enviroment
test_count= directed_count + random_count;
and then drv.run (test_count);
driver will know what he will receive exactly */

alu_transaction tr; //transaction handle

repeat (count) begin

    gen2drv.get(tr); //receive transactions from generator
    //this means, take transactions from gen2drv mailbox and put it in
tr

    //we will use negative edge because DUT reads and output happens
@posedge

    /*this means:
    1.driver drives at negedge
    2.DUT samples/calculates @posedge
    3.monitor samples after posedge
    */
    @(negedge vif.clk);
    //put the value of tr on real signal that is inside interface
    vif.A <= tr.A;
    vif.B <= tr.B;
    vif.ALU_en <= tr.ALU_en;
    vif.a_en <= tr.a_en;

```

```
        vif.a_op    <= tr.a_op;  
        vif.b_en    <= tr.b_en;  
        vif.b_op    <= tr.b_op;  
    end  
endtask  
endclass
```

4.7 Monitor Class

The monitor samples the interface signals after the DUT output is updated. It makes an observed transaction containing the applied inputs and actual output, then sends it to the scoreboard by the mon2scb mailbox.

4.7.1 Monitor Class Code

```
class alu_monitor;
/*monitor do the following
  1.Read DUT/Interface signals
  2.Store them in transaction
  3.Send transaction to scoreboard
*/
virtual alu_if vif; //handle for real interface
//monitor has a reference to the real alu_if instance created in tb_top

mailbox #(alu_transaction) mon2scb; //monitor to scoreboard mailbox
//monitor put inside observed transactions

//constructor new
//called from enviroment when it makes mon=new(vif,mon2scb);
//it takes 2 values 1.virtual interface handle, 2.mailbox connected to
scoreboard
function new(virtual alu_if vif, mailbox #(alu_transaction) mon2scb);

    this.vif      = vif; //store virtual interface inside monitor (so
monitor can read signals using vif.A,..etc)
    this.mon2scb = mon2scb; //store mailbox inside monitor so monitor can
send trans. to scoreboard

endfunction

task run(int count); //takes parameter count, no.of transactions that
monitor will observe

    alu_transaction tr; //transaction variables
    //new transaction based on readings from interface

    repeat (count) begin

        //wait on +ve edge because the design output happens at posedge
clk
        //normally, we will monitor @posedge to see new output
        @(posedge vif.clk);
        #1ps; //delay after positive edge to avoid race condition
        //at same positive edge DUT updates C output using non-blocking
assignment
```

```

        //so if monitor read the C at same clock, may read the old value
        //so we will wait for 1 picosecond after clock edge and then read

        tr = new(); //new object of alu_transaction
        //read the signal from interface and put it inside the transaction
        tr.A      = vif.A;
        tr.B      = vif.B;
        tr.ALU_en = vif.ALU_en;
        tr.a_en   = vif.a_en;
        tr.a_op   = vif.a_op;
        tr.b_en   = vif.b_en;
        tr.b_op   = vif.b_op;

        // read output from DUT and store it inside actual
        //actual = real value output from DUT
        //scoreboard compares actual vs expected
        tr.actual = vif.C;

        //send the transactions to scoreboard
        mon2scb.put(tr);

    end
endtask
endclass

```


4.8 Scoreboard Class

The scoreboard is in charge for checking the exactness of the DUT. It holds a golden model that calculates the expected ALU result according to the requirement. The scoreboard compares the expected result with the actual DUT output and reports PASS or FAIL for each transaction.

4.8.1 Scoreboard Class Code

```
class alu_scoreboard;

    /*it does the following
    1.Calculate expected output
    2.Compare expected with actual DUT output
    3.Print Pass or Fail*/

    mailbox #(alu_transaction) mon2scb; //mailbox from monitor to scoreboard

    int pass_count; //count passed transactions
    int fail_count; //count failed transactions

    //constructor new
    // when enviroment call scoreboard, scb=new(mon2scb);
    //means that scoreboard will take mailbox that received from monitor
    function new(mailbox #(alu_transaction) mon2scb);

        this.mon2scb = mon2scb; //store mailbox inside scoreboard
        pass_count    = 0;
        fail_count    = 0;

    endfunction

    //function extension 5-bits to 6-bits signed value
    function automatic logic signed [5:0] ext5(input logic [4:0] value);

        ext5 = $signed({value[4], value});

    endfunction

    //function golden_model
    //calculate output C as expected from specs
    //takes transactions tr
    //can see tr.A,tr.B,...till tr.b_op
    function automatic logic signed [5:0] golden_model(alu_transaction tr);

        if (!tr.ALU_en) begin
            //if ALU_en = 0 then expected output = 0
            golden_model = '0;
        end
    endfunction
endclass
```

```

end
else begin

    case ({tr.a_en, tr.b_en}) //Concatenation

        2'b10: begin
            case (tr.a_op)
                3'd0:    golden_model = tr.A + tr.B;
                3'd1:    golden_model = tr.A - tr.B;
                3'd2:    golden_model = ext5(tr.A ^ tr.B);
                3'd3:    golden_model = ext5(tr.A & tr.B);
                3'd4:    golden_model = ext5(tr.A & tr.B);
                3'd5:    golden_model = ext5(tr.A | tr.B);
                3'd6:    golden_model = ext5(~(tr.A ^ tr.B));
                default: golden_model = '0;
            endcase
        end

        2'b01: begin
            case (tr.b_op)
                2'd0:    golden_model = ext5(~(tr.A & tr.B));
                2'd1:    golden_model = tr.A + tr.B;
                2'd2:    golden_model = tr.A + tr.B;
                default: golden_model = '0;
            endcase
        end

        2'b11: begin
            case (tr.b_op)
                2'd0:    golden_model = ext5(tr.A ^ tr.B);
                2'd1:    golden_model = ext5(~(tr.A ^ tr.B));
                2'd2:    golden_model = tr.A - 6'sd1;
                2'd3:    golden_model = tr.B + 6'sd2;
                default: golden_model = '0;
            endcase
        end

        //default enable mode if a_en=0, b_en=0 -> No operations
selected

        //No operations selected
        default: begin
            golden_model = '0;
        end

    endcase

end

```

```

endfunction

task run(int count);
    //takes count, means no.of transactions from scoreboard is same test cases
    //generator send and monitor observed

    alu_transaction tr; //transaction variable

    repeat (count) begin

        mon2scb.get(tr); //get observed transaction from monitor

        tr.expected = golden_model(tr); //calculate expected output
        // so here takes same transaction input/control signals
        // & send it to the golden_model

        if (tr.actual === tr.expected) //comparison between expected and
actual
        begin
            pass_count++;
            $display("[SCB] PASS : %s", tr.to_string());
        end
        else begin
            fail_count++;
            $display("[SCB] FAIL : %s", tr.to_string());
        end
    end
    //after finishing call function report to print summary of results
    report();
endtask

function void report();
    $display("=====");
    $display("ALU SCOREBOARD REPORT");
    $display("PASS   : %0d", pass_count);
    $display("FAIL    : %0d", fail_count);
    $display("TOTAL   : %0d", pass_count + fail_count);
    $display("=====");
endfunction
endclass

```

4.9 Environment Class

The Environment builds, connects, and runs all verification components. It creates the generator, driver, monitor, and scoreboard, connects them using mailboxes, applies reset, and runs the test.

4.9.1 Environment Class Code

```
class alu_environment;
    /*enviroment do these things:
    1.Build components
    2.connect components
    3.Run components
    */

    //so it created generator, driver, monitor and scoreboard and connect them
    by mailboxes and vif
    //then run them all with each other

    virtual alu_if vif; //handle for real interface
    //it need to handle reference to send it to driver and monitor
    //so it receive interface from tb_top and then gives it to the components
    that need it

    mailbox #(alu_transaction) gen2drv; //mailbox between generator to driver
    mailbox #(alu_transaction) mon2scb; //mailbox between monitor to
    scoreboard

    //component handles
    alu_generator  gen;
    alu_driver     drv;
    alu_monitor    mon;
    alu_scoreboard scb;

    //counters
    int random_count; //generator will make 100 random tests
    int directed_count; //generator will ake 18 directed tests
    int test_count; //total = random + directed

    //constructor
    //almost called from tb_top as env=new(alu_vif,100);

    function new(virtual alu_if vif, int random_count = 100);

        this.vif          = vif; //store interface hand inside env
        this.random_count = random_count; //store random tests inside env

        directed_count = 18; //same as send_directed() in generator
```

```

    test_count      = random_count + directed_count;

    gen2drv = new(); //mailbox object
    mon2scb = new(); //mailbox object

    gen = new(gen2drv, random_count);
    drv = new(vif, gen2drv);
    mon = new(vif, mon2scb);
    scb = new(mon2scb);
endfunction
task run();
    //firstly, call reset task of driver
    //bec before start test, DUT must be in a known state
    drv.reset();
    fork
        gen.run();
        drv.run(test_count);
        mon.run(test_count);
        scb.run(test_count);
    join
    //after finished all components
    //env take a look on no.of failures in scoreboard
    if (scb.fail_count != 0) begin
        $fatal(1, "ALU test failed, fail_count = %0d", scb.fail_count);
    end
endtask
endclass

```

4.10 ALU Package

```
`timescale 1ns/1ps

//Package for all classes
package alu_pkg;
    `include "alu_transaction.sv"
    `include "alu_generator.sv"
    `include "alu_driver.sv"
    `include "alu_monitor.sv"
    `include "alu_scoreboard.sv"
    `include "alu_environment.sv"
endpackage
```

4.11 Top Module

The top module instantiates the DUT, interface, clock generator, and verification environment. It starts the simulation by creating the environment object and calling the environment run task.

4.11.1 Top Module Code

```
`timescale 1ns/1ps

module tb_top;

    /*combine 3 main things
    1.Clock
    2.Interface
    3.DUT + Enviroment
    */
    import alu_pkg::*;
    logic clk;
    alu_if alu_vif(clk); //interface instance alu_vif and send clock to it
    alu_environment env; //enviroment handle
    //DUT instantiation
    alu dut (
        .A      (alu_vif.A), //port A in Dut is connect to Signal A inside
interface
        .B      (alu_vif.B),
        .a_en   (alu_vif.a_en),
        .a_op   (alu_vif.a_op),
        .b_en   (alu_vif.b_en),
        .b_op   (alu_vif.b_op),
        .rst_n  (alu_vif.rst_n),
        .clk    (alu_vif.clk),
        .ALU_en (alu_vif.ALU_en),
        .C      (alu_vif.C)
    );
    //clock generator
    initial begin

        clk = 1'b0;
        forever #5 clk = ~clk;

    end
    initial begin

        env = new(alu_vif, 100); //create object from alu_enviroment
        //send 2 things 1.alu.vif which real interface instance
        //this instance goes to env and env send it to driver and monitor as
virtual interface
        //2.random counts =100
```

```
$display("=====");
$display("Starting ALU test");
$display("=====");
env.run();//start real test
/*inside enviroment
1.drv.reset()
2.for generator,driver,monitor,scoreboard
3.wait until all finish
4.check fail_count*/
$display("=====");
$display("ALU test completed");
$display("=====");
$finish;
end
endmodule
```


5. Coverage Results and Analysis

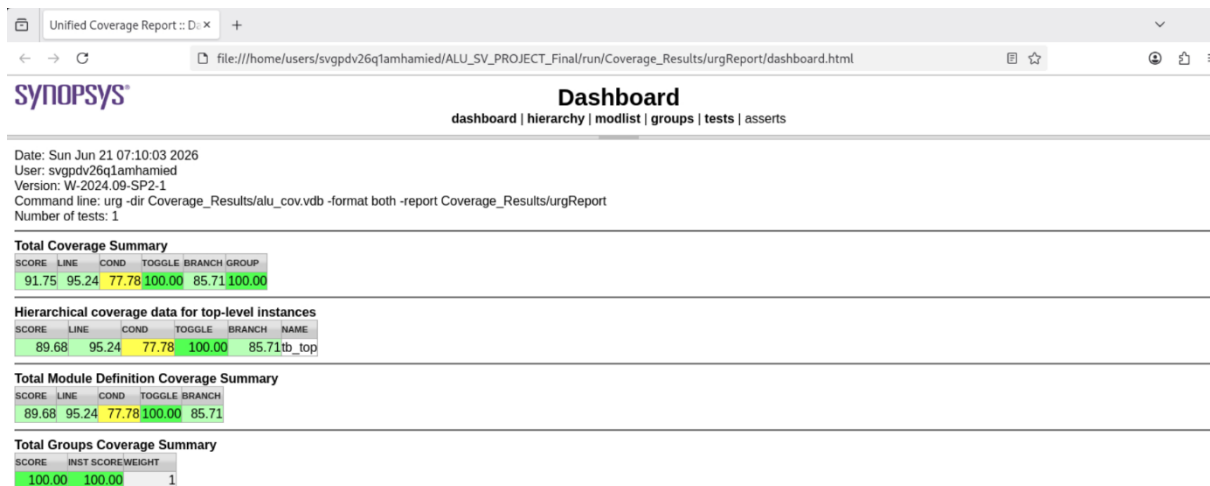


FIGURE 2: TOTAL COVERAGE SUMMARY

5.1 Coverage Results

Coverage collection was made to evaluate how totally the ALU design and the verification environment were exercised through simulation. The coverage analysis contains functional coverage, assertion coverage, and code coverage. The main goal of this section is to show whether the verification plan was achieved and to clarify any lasting coverage gaps.

The simulation was done using two kinds of tests: directed tests and constrained-random tests. Directed tests made sure that all the important Arithmetic Logic Unit operation modes and boundary cases were checked. Constrained-random tests were used to make sure that lots of things were tried out. This means trying inputs and control combinations for the Arithmetic Logic Unit as this helps to increase confidence, in the simulation.

1. Functional Coverage Analysis

Functional coverage was applied using a covergroup inside the ALU interface. The aim of the functional coverage model is to check whether the significant scenarios from the verification plan were exercised during simulation.

The covergroup included the following overpoints:

Coverpoint	Purpose
cp_rst_n	Covers reset asserted and reset released states
cp_ALU_en	Covers ALU enabled and disabled states
cp_enable_mode	Covers all enable combinations: no-operation, A-only, B-only, and both-enabled modes
cp_a_op_a_only	Covers all legal a_op values in A-only mode
cp_b_op_b_only	Covers all legal b_op values in B-only mode
cp_b_op_both	Covers all b_op values in both-enabled mode
cp_A_values	Covers important values of operand A: minimum, zero, maximum, negative, and positive

cp_B_values	Covers important values of operand B: minimum, zero, maximum, negative, and positive
-------------	--

The functional coverage reached 100%. This means that all planned functional coverage bins were hit successfully. Thus, the verification environment successfully covered all required functional scenarios defined in the verification plan.

The illegal opcode bins, such as a_op = 7 in A-only mode and b_op = 3 in B-only mode, were not hit. This is predictable because these values are classified as illegal cases and are intentionally disallowed during legal constrained-random testing.

5.2 Assertion Coverage Analysis

Assertions were added inside the ALU interface to check important design rules during simulation. These assertions confirm reset behavior, ALU disable behavior, illegal opcode prevention, and output range correctness.

The following assertions were used:

Assertion	Purpose	Result
reset_p	Checks that output C is zero when reset is active	Passed
alu_disable_p	Checks that output C becomes zero when ALU_en is low	Passed
illegal_aop_p	Checks that a_op = 7 is not used in A-only mode	Passed
illegal_bop_p	Checks that b_op = 3 is not used in B-only mode	Passed
range_p	Checks that output C remains within the signed 6-bit range	Passed

Assertion coverage reached 100%, and no assertion failures were reported. This confirms that the major protocol and control rules were respected during simulation.

5.3 Code Coverage Analysis

Code coverage was collected using Synopsys VCS and URG. The coverage report showed the following results:

Coverage Metric	Result
Total Coverage Score	91.75%
Line Coverage	95.24%
Condition Coverage	77.78%
Toggle Coverage	100.00%
Branch Coverage	85.71%
Group Coverage	100.00%

Toggle coverage reached 100%, which means that all monitored design signals toggled between 0 and 1 during simulation. Group coverage also reached 100%, which confirms that the functional coverage model was fully covered.

Line coverage reached 95.24%, which is a strong result. The uncovered lines mainly correspond to default branches for illegal opcode values or failure-handling code inside the testbench.

Branch coverage reached around 85%. The uncovered branches are mainly related to default cases and error/failure paths that are not expected to occur during legal testing.

Condition coverage reached 77.78%. This is lower than the other metrics mainly because some conditions are part of priority if-else logic. In such logic, some combinations become structurally unreachable because earlier conditions already capture those cases.

5.4 Coverage Gap and Bug Analysis

This section explains the uncovered coverage items and whether they represent real design bugs.

5.4.1 Line Coverage Gap

The line coverage did not reach 100% because some lines were not executed during simulation. The main uncovered lines are related to default branches inside the DUT operation selection logic.

Examples of such cases include:

- Default branch for a_op in A-only mode.
- Default branch for b_op in B-only mode.

These branches correspond to illegal operation values such as:

- a_op = 7 in A-only mode
- b_op = 3 in B-only mode

These values are illegal according to the verification plan and are intentionally not generated during legal testing. Thus, the missed line coverage does not indicate a design bug.

Some uncovered lines also exist in the testbench package. These lines are related to failure-handling paths, such as:

- randomization failure handling
- scoreboard mismatch handling
- fatal error handling when fail_count is not zero

Since the simulation completed successfully and all transactions passed, these failure-handling lines were not expected to execute.

5.4.2 Branch Coverage Gap

The branch coverage did not reach 100% because some branches represent illegal or default cases. These branches are intentionally avoided by the constrained-random stimulus.

For example, the legal stimulus prevents invalid opcode combinations such as:

- A-only mode with a_op = 7
- B-only mode with b_op = 3

The verification environment treats these cases as illegal. The assertions also check that these illegal cases do not occur during normal operation.

Therefore, hitting these branches would require generating illegal stimulus. This is not part of the normal legal verification test and may cause assertion failures. For this reason, the uncovered branches are not considered functional bugs.

5.4.3 Condition Coverage Gap

Condition coverage is lower than the other metrics because some conditions are part of priority if-else logic.

For example, the DUT checks the enable modes using conditions similar to:

```
if (!in_alu_en)
else if (in_a_en && !in_b_en)
else if (!in_a_en && in_b_en)
else if (in_a_en && in_b_en)
else
```

In this code, once a condition is true, the following conditions are not evaluated for that same transaction. Therefore, some combinations may not be counted as covered by the condition coverage tool, even though the corresponding functional scenario was already tested in another branch.

This means the condition coverage gap is mainly caused by the structure of the priority logic, not by a missing functional scenario.

Some condition coverage gaps also come from testbench failure conditions, such as:

- if (tr.actual === tr.expected)

The false branch of this condition was not hit because all DUT outputs matched the expected outputs. This is a good result, not a bug.

Similarly:

- if (scb.fail_count != 0)

The true branch was not hit because the scoreboard reported zero failures.

5.4.3 Bug Analysis Summary

No functional bugs were detected during simulation.

The scoreboard reported:

PASS : 118

FAIL : 0

TOTAL : 118

This means that all generated transactions produced the expected DUT output according to the golden reference model.

The remaining uncovered coverage items are classified as justified coverage gaps, not functional bugs.

Coverage Gap	Reason	Bug
Default a_op branch	Represents illegal a_op = 7 in A-only mode	No
Default b_op branch	Represents illegal b_op = 3 in B-only mode	No
Scoreboard failure branch	Not reached because all comparisons passed	No
Environment fatal branch	Not reached because fail_count = 0	No
Randomization failure branch	Not reached because randomization succeeded	No
Some condition combinations	Structurally unreachable due to priority if-else logic	No

Thus, the uncovered coverage items are not considered design bugs. They are either illegal/default cases, structurally inaccessible conditions, or failure-handling paths that should not be executed in a successful legal simulation.

```
[SCB] PASS : A=4 B=-9 ALU_en=1 a_en=1 a_op=4 b_en=0 b_op=1 expected=4 actual=4
[SCB] PASS : A=-15 B=11 ALU_en=1 a_en=1 a_op=6 b_en=0 b_op=2 expected=5 actual=5
[SCB] PASS : A=-14 B=-3 ALU_en=1 a_en=0 a_op=7 b_en=0 b_op=3 expected=0 actual=0
[SCB] PASS : A=4 B=6 ALU_en=1 a_en=0 a_op=3 b_en=0 b_op=3 expected=0 actual=0
[SCB] PASS : A=-5 B=-11 ALU_en=1 a_en=1 a_op=5 b_en=0 b_op=1 expected=-1 actual=-1
[SCB] PASS : A=-9 B=12 ALU_en=1 a_en=0 a_op=0 b_en=1 b_op=1 expected=3 actual=3
[SCB] PASS : A=12 B=10 ALU_en=0 a_en=1 a_op=1 b_en=1 b_op=1 expected=0 actual=0
[SCB] PASS : A=7 B=4 ALU_en=1 a_en=0 a_op=6 b_en=1 b_op=1 expected=11 actual=11
[SCB] PASS : A=1 B=6 ALU_en=1 a_en=0 a_op=1 b_en=1 b_op=2 expected=7 actual=7
[SCB] PASS : A=11 B=0 ALU_en=1 a_en=1 a_op=1 b_en=1 b_op=2 expected=10 actual=10
[SCB] PASS : A=-11 B=9 ALU_en=1 a_en=0 a_op=2 b_en=1 b_op=2 expected=-2 actual=-2
[SCB] PASS : A=7 B=8 ALU_en=1 a_en=1 a_op=5 b_en=0 b_op=0 expected=15 actual=15
[SCB] PASS : A=7 B=10 ALU_en=1 a_en=0 a_op=1 b_en=1 b_op=2 expected=17 actual=17
[SCB] PASS : A=9 B=-14 ALU_en=1 a_en=0 a_op=3 b_en=0 b_op=2 expected=0 actual=0
[SCB] PASS : A=-14 B=-14 ALU_en=1 a_en=0 a_op=0 b_en=1 b_op=0 expected=13 actual=13
[SCB] PASS : A=-1 B=3 ALU_en=1 a_en=1 a_op=5 b_en=1 b_op=0 expected=-4 actual=-4
[SCB] PASS : A=8 B=8 ALU_en=1 a_en=0 a_op=6 b_en=1 b_op=0 expected=-9 actual=-9
[SCB] PASS : A=-14 B=-8 ALU_en=1 a_en=0 a_op=2 b_en=0 b_op=1 expected=0 actual=0
[SCB] PASS : A=7 B=-1 ALU_en=1 a_en=1 a_op=2 b_en=1 b_op=2 expected=6 actual=6
[SCB] PASS : A=-16 B=4 ALU_en=1 a_en=1 a_op=7 b_en=1 b_op=3 expected=6 actual=6
[SCB] PASS : A=-16 B=-16 ALU_en=1 a_en=1 a_op=7 b_en=1 b_op=2 expected=-17 actual=-17
=====
ALU SCOREBOARD REPORT
PASS : 118
FAIL : 0
TOTAL : 118
=====
ALU test completed
=====
$finish called from file "../tb/tb_top.sv", line 71.
$finish at simulation time 1225001

VCS Coverage Metrics: during simulation line, cond, branch, tgl was monitored
-----
Coverage status: End of All Coverages ...

VCS Simulation Report
```

FIGURE 3: TRANSACTIONS

6. Simulation Result

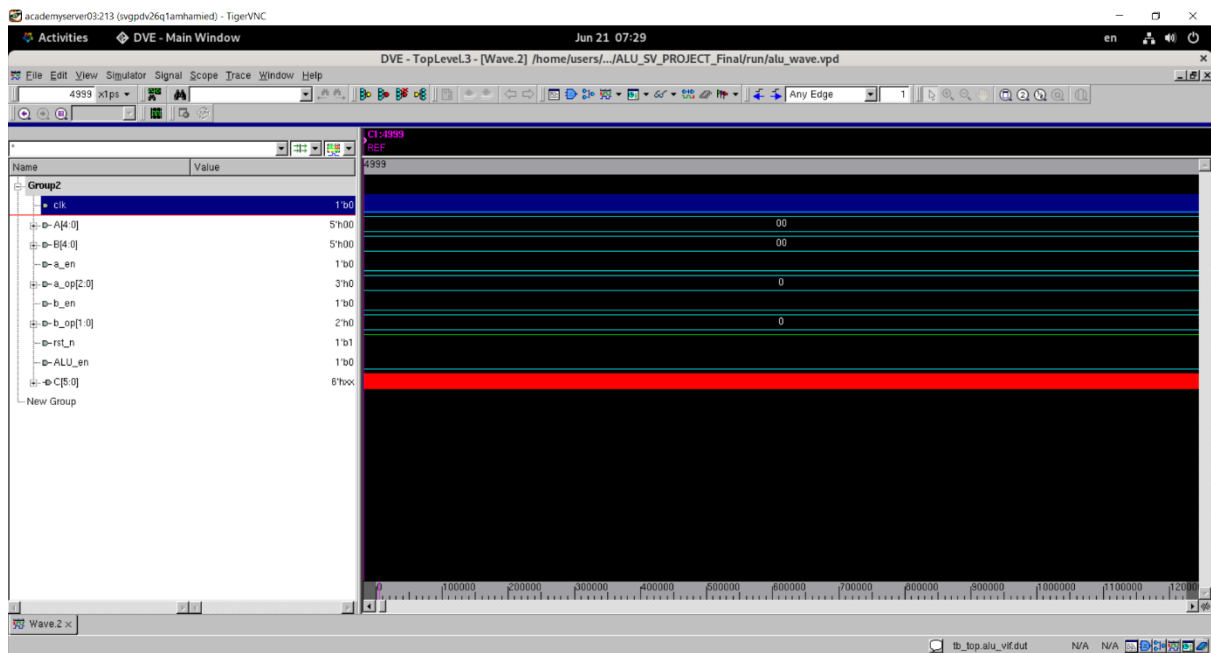


FIGURE 4:SIMULATION RESULTS

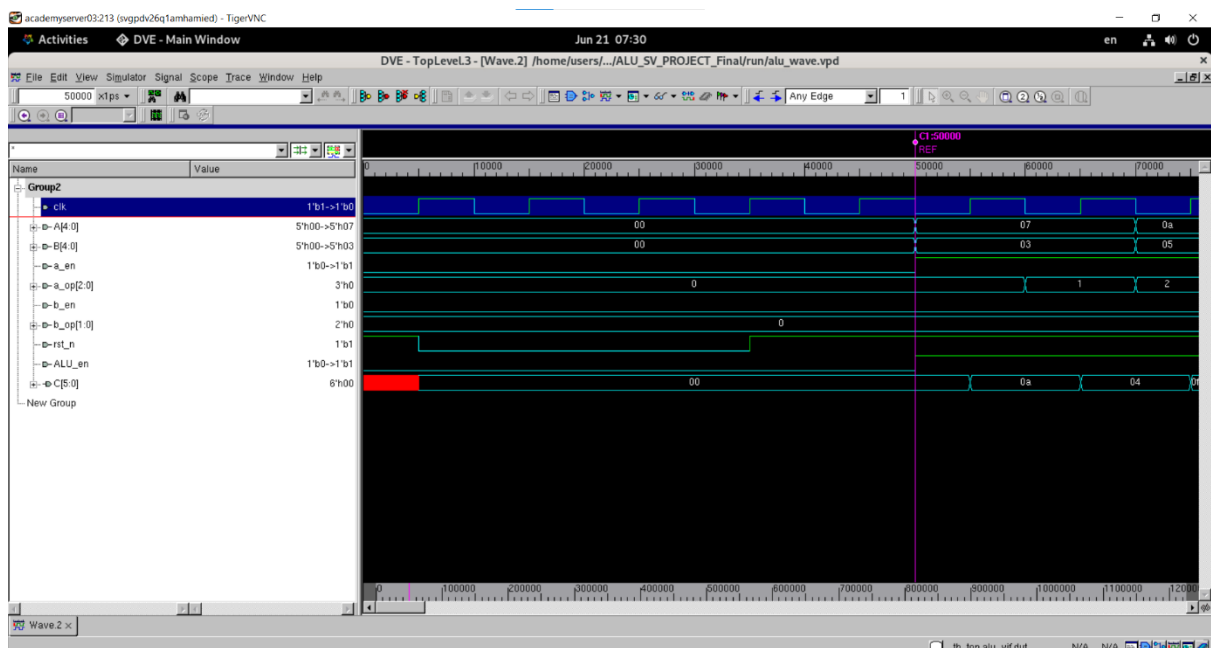


FIGURE 5: SIMULATION RESULTS

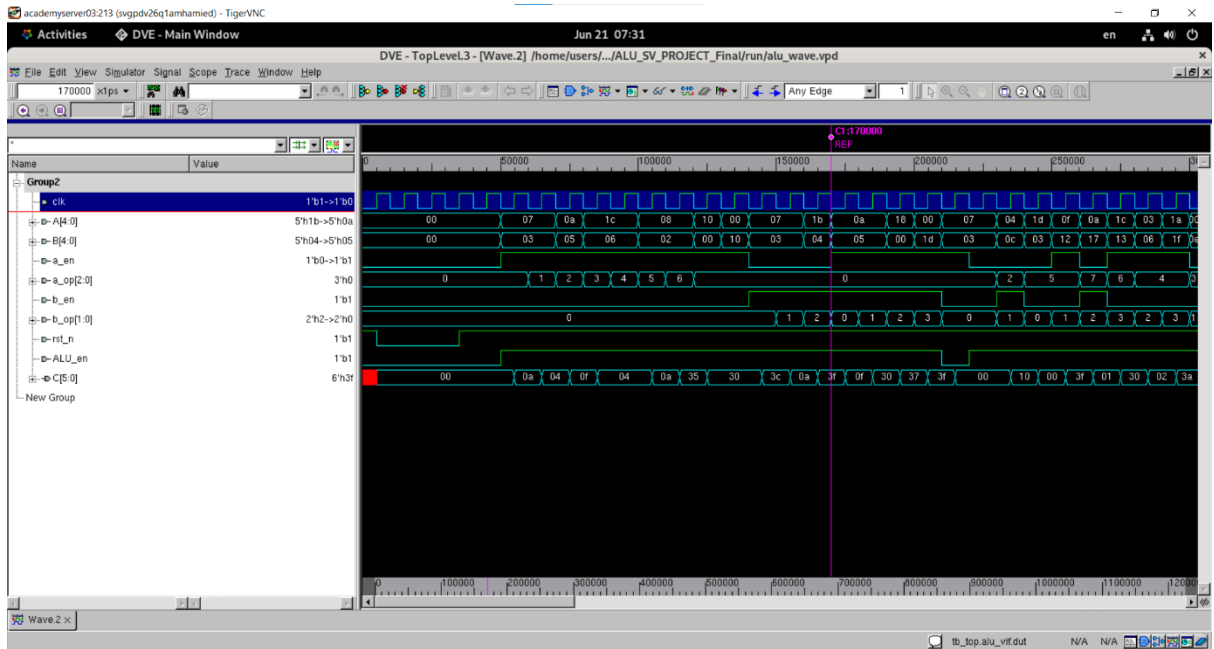


FIGURE 6: SIMULATION RESULTS

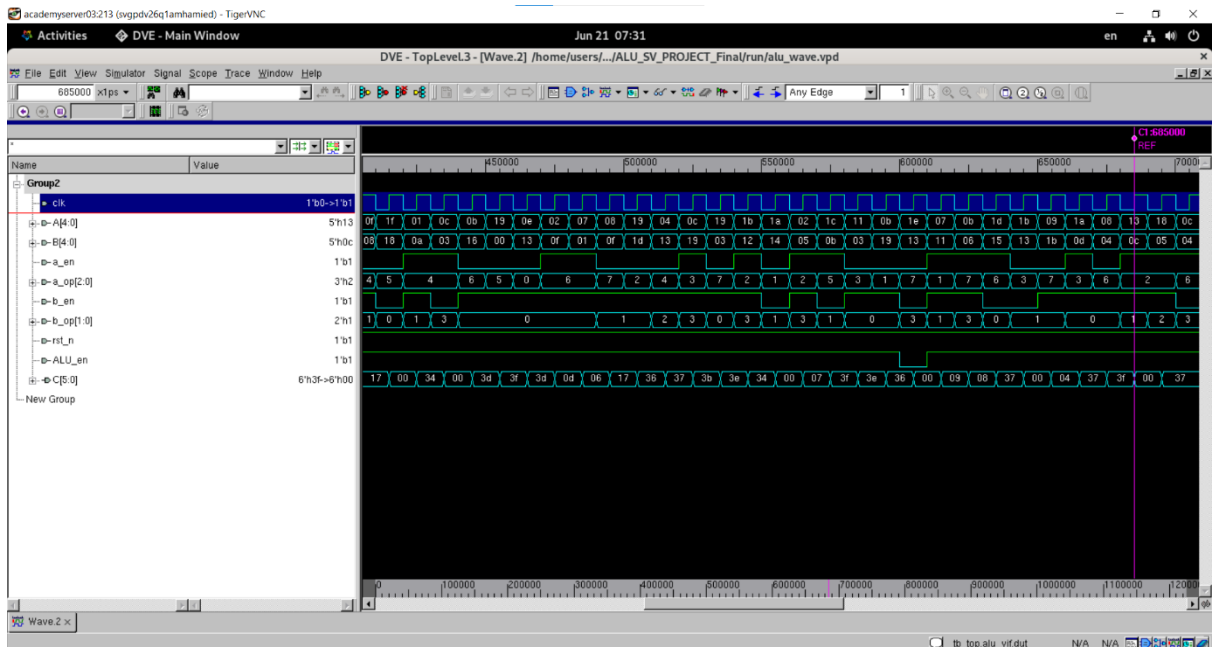


FIGURE 7: SIMULATION RESULTS

7. Conclusion

A SystemVerilog OOP-based verification environment was used to successfully validate the ALU design. A generator, driver, monitor, scoreboard, interface, assertions, and functional coverage were all part of the environment.

Constrained-random tests were employed to boost verification confidence, while directed tests ensured coverage of all authorized operation modes and significant boundary values. The scoreboard revealed 118 successful transactions with no failures when comparing the DUT output to a golden reference model.

Both assertion coverage and functional coverage attained 100%. In several measurements, code coverage fell short of 100%; the remaining gaps were associated with failure-handling paths, structurally unreachable scenarios, and illegal/default cases. These holes are not regarded as functional defects.

The ALU is deemed functionally sound based on the simulation results, scoreboard checking, assertions, and functional coverage.